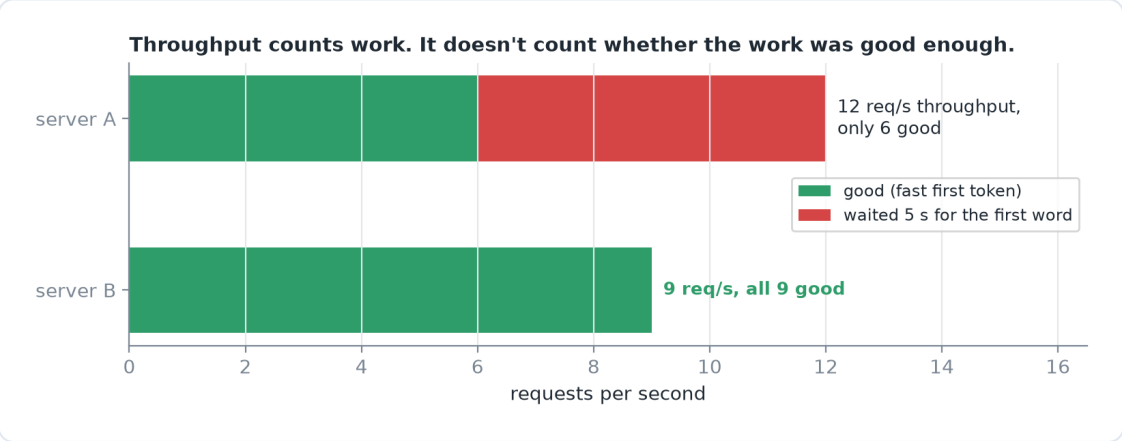
23 · Goodput, SLOs & Failure Rates

Source: fanout.sh / inference-eng / 04-06 · ~6 min

IN SHORT

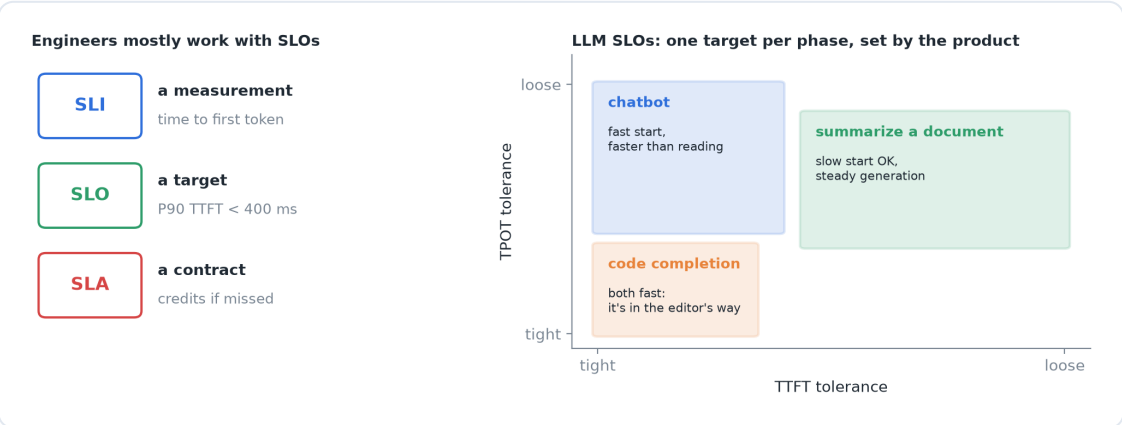
Throughput counts work, not whether the work was good enough. An **SLO** turns a measurement into a target set by the **product** (e.g. P90 TTFT < 400 ms), usually one per phase. A request is **good** if it meets **every** target; **goodput** = good requests per second (vLLM example: **4.23** req/s throughput, **1.69** goodput). Sweep load and throughput keeps rising while goodput **peaks and falls**: operate near the goodput peak with headroom. **Failures count as bad**, always, or dropping the slowest requests makes P99 look great. An **error budget** (99% SLO → 10,000 bad per million) and its **burn rate** (10× → gone in **3 days**) turn this into operations.

1. Throughput isn't the goal



Server A completes 12 req/s but half its users waited 5 s for the first word. Server B completes 9, all fast. B serves more happy users.

2. SLI, SLO, SLA



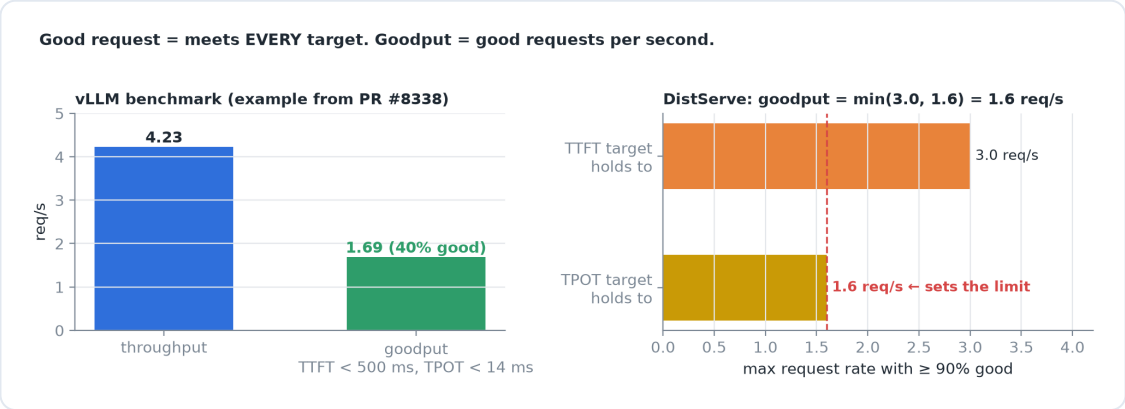
Left: the three terms from site reliability engineering. Right: targets per product (after the DistServe paper's workload table).

- A useful LLM SLO combines targets for **both phases** (TTFT and TPOT).
- **Chatbot:** fast first token, token rate faster than people read. **Code completion:** both fast, it's in the editor's way. **Summarizing a long document:** slow start is fine, generation must be steady.

> **KEY IDEA**
>
> The targets come from the **product**, not from the hardware.

3. Goodput

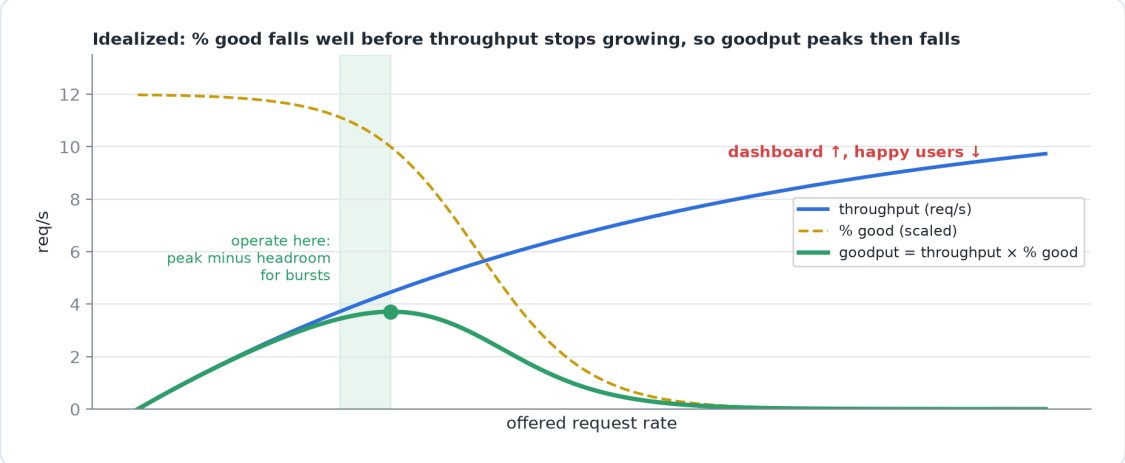
A request is **good** if TTFT < its limit **and** TPOT < its limit. **Goodput** = good requests completed per second.



Left: vLLM's benchmark reports goodput when you pass targets; in the pull request that added it, 4.23 req/s throughput was only 1.69 req/s goodput. Right: DistServe's stricter version is the highest request rate at which ≥ 90% of requests are still good; the weakest target sets it.

```
vllm bench serve ... --goodput ttft:500 tpot:14
```

4. Throughput vs goodput



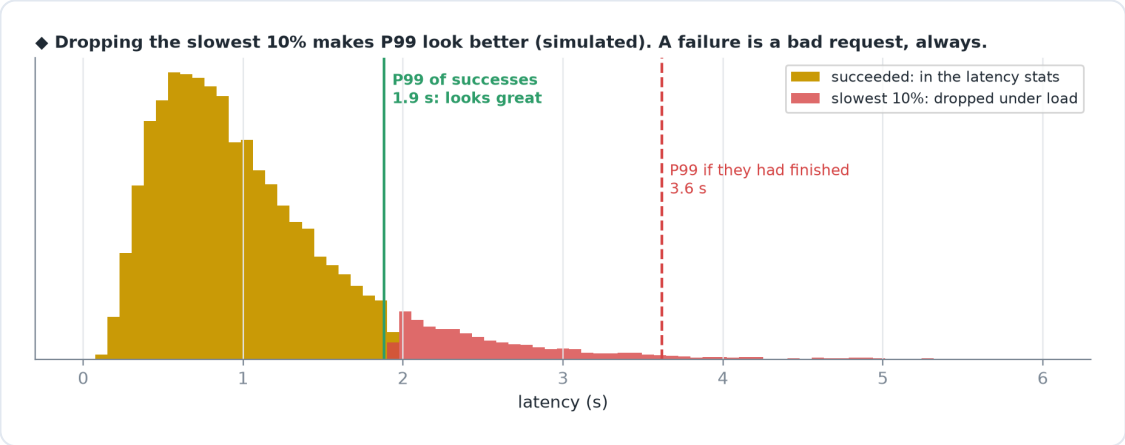
As the request rate rises, throughput climbs then flattens as the GPU fills. The share of good requests starts falling much earlier, as queues and batching slow everyone down. Their product, goodput, peaks and then falls.

CAUTION

Past the goodput peak, more load makes the dashboard go up while the number of happy users goes down. Operate **near the goodput peak with headroom for bursts**, not at the throughput maximum.

5. Failures

Some requests don't just run slowly, they fail: **server errors (5xx), timeouts, rejections when overloaded (429 / load shedding), out-of-memory crashes, replies cut off at a length limit**.

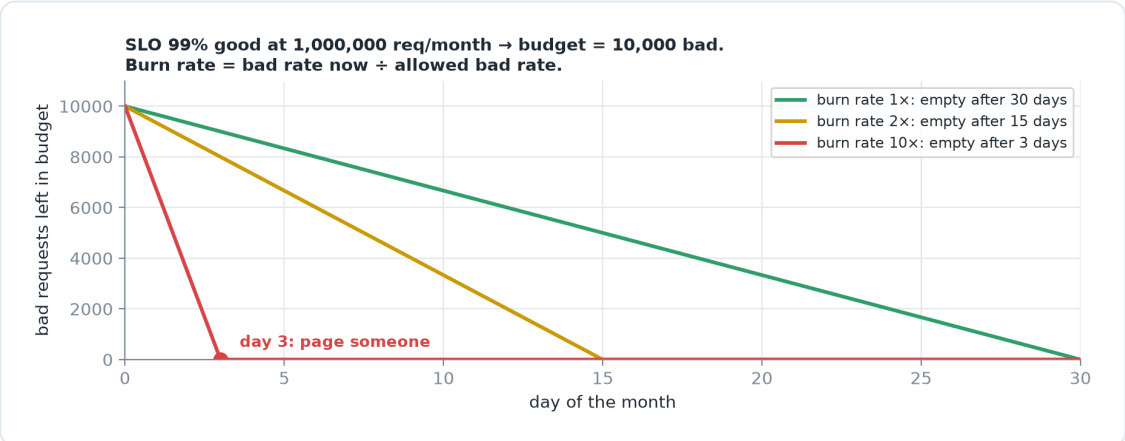


Latency stats are usually computed over successes only. A server that drops its slowest 10% under load suddenly shows a great P99, because the worst requests left the data.

KEY IDEA

A failure counts as a **bad request, every time**. Report the **failure rate next to latency** and include failures in goodput.

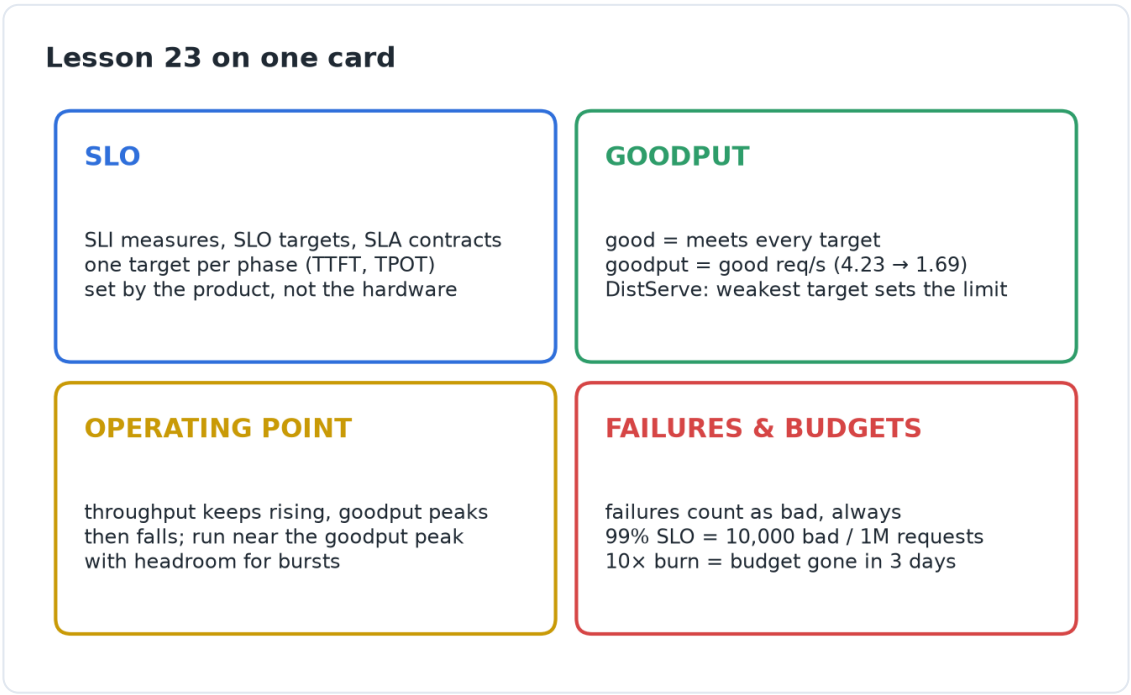
6. Error budgets



An SLO of 99% good allows 1% bad: at a million requests a month, 10,000 bad requests.

- **Spend slowly:** you can ship risky changes. **Burn fast:** freeze and fix.
- **Alert on the burn rate** (bad rate now ÷ allowed bad rate). Burning **10× too fast** empties a month's budget in about **3 days**: page someone.

7. Recap



Interview cheat sheet

Revise from these pages alone. ◆ = my addition beyond the video; everything else is from the lesson. TTFT/TPOT definitions are in 20; the queueing knee behind the falling good share in 21; P99 traps in 22.

Say it in 30 seconds

> *Throughput counts work, not good work, so I'd set SLOs: targets on SLIs like P90 TTFT under 400 ms, one per phase, chosen by the product. A request is good only if it meets every target, and goodput is good requests per second: vLLM's example showed 4.23 req/s throughput but 1.69 goodput. DistServe defines it as the max rate with 90% good, set by the weakest target. As load rises goodput peaks and falls while throughput keeps climbing, so operate near the peak with headroom. Failures always count as bad, or P99 flatters you. A 99% SLO gives a 1% error budget, and you alert on burn rate: 10× burns a month in 3 days.*

Only numbers worth memorizing

- **Error budget:** 99% SLO at 1M requests/month = **10,000 bad**; burn rate **10× → 3 days**.

Derive, don't memorize

1. Goodput = throughput × share of requests meeting every target

$$\text{A}: 12 \times 0.5 = 6 < \text{B}: 9 \times 1 = 9 \qquad \text{vLLM}: \frac{1.69}{4.23} \approx 40\%\ \text{good}$$

> **INTUITION**
>
> "Every target" is an AND: ◆ if TTFT and TPOT each pass 80% of requests independently, only ~64% are good.

2. DistServe goodput: the weakest target sets the limit

$$\text{goodput} = \min_{\text{targets}} (\text{max rate with} \geq 90\%\ \text{good}) = \min(3.0,\ 1.6) = 1.6\ \text{req/s}$$

> **INTUITION**
>
> To raise it, fix the binding target (here TPOT); improving the other one buys nothing.

3. Error budget and burn rate

$$\text{budget} = 10^6 \times 1\% = 10000 \qquad \text{burn} = \frac{\text{bad rate now}}{\text{allowed bad rate}} \qquad \text{time left} = \frac{30\ \text{days}}{10} = 3\ \text{days}$$

4. Dropping failures shifts the percentile ◆

If the slowest 10% fail and vanish, the survivors' P99 is the full distribution's P(0.99 × 90) = P89.1: below where the tail starts.

$$\text{simulated}: 1.9\ \text{s}\ (\text{successes only}) \quad \text{vs} \quad 3.6\ \text{s}\ (\text{had they finished})$$

Target per product (after DistServe)

App	TTFT	TPOT
Chatbot	Tight	Faster than reading
Code completion	Tight	Tight
Summarize a document	Loose	Steady

Symptom → diagnosis

Symptom	Diagnosis
Throughput up, complaints up	Past the goodput peak: share of good requests collapsing
P99 improved exactly when load spiked	Failures dropped from latency stats
Goodput stuck despite a faster prefill	TPOT is the binding target (derivation 2)
Budget half gone in the first week	Burn rate ~2×: slow risky launches
High 429 rate, great latency	Load shedding: rejections are bad requests

Rapid-fire Q&A

Question	Crisp answer
SLI vs SLO vs SLA?	Measurement, target, contract with consequences.
Who sets SLO targets?	The product, not the hardware.
What is a good request?	One that meets every target, TTFT and TPOT.
Where should you operate?	Near the goodput peak, with headroom for bursts.
How do failures enter the metrics?	As bad requests: failure rate next to latency, inside goodput.
What do you alert on?	Burn rate, not single bad requests.

TRAP

- Tuning for max throughput instead of goodput.
- Computing latency percentiles over successes only.
- Treating timeouts, 429s or truncated replies as "not latency problems".